Módulo 00 · Configuración y alias

Curso de Git Avanzado · Apuntes de estudio · Laboratorio: `laboratorio/m00-config`

1. Niveles de configuración
2. Configuración por repo (local)
3. Alias: `st`, `lg`, `last`
4. Alias `hist` y formatos de log
5. Ajustes globales recomendados
6. Alias de shell (`!`): `ramas`
7. `includeIf`: trabajo vs. personal
8. `git config --file` y dónde viven los archivos
9. Reto: `deshacer` y `limpiar-ramas`
10. `restore` vs. `reset`
11. `HEAD~N` y `HEAD^`
12. Chuleta final y repaso

1. Niveles de configuración

Nivel	Flag	Archivo	Alcance
Sistema	`--system`	`<git>/etc/gitconfig`	Todos los usuarios del equipo
Global	`--global`	`~/.gitconfig`	Tu usuario
Local	`--local` (default)	`.git/config`	Solo ese repo
Worktree	`--worktree`	`.git/config.worktree`	Solo un worktree

Regla: gana el nivel **más específico** (worktree > local > global > sistema), y cada clave se resuelve por separado. Puedes tener `user.name` local y `user.email` global a la vez.

```
git config --list --show-origin      # todo + de qué archivo viene
git config --show-scope user.name    # ¿qué nivel define este valor?
git config --global --edit           # abre ~/.gitconfig en el editor
```

Lo que viste: `core.autocrlf` estaba en *system* (`true`) y en *local* (`false`). En ese repo ganó el local.

2. Configuración por repo (local)

```
git config user.name "Jesus Practica"     # sin flag = --local
git config --show-scope user.name         # local   Jesus Practica
git config --global user.name             # el global sigue intacto
git log -1                                # el nuevo commit usa el autor local
```

Sobrescribir y borrar claves

Comando	Efecto
`git config clave valor`	Crea la clave o la **sobrescribe** si ya existe
`git config --add clave valor`	Agrega un segundo valor (clave multivalor)
`git config --unset clave`	Borra la clave
`git config --unset-all clave`	Borra todos los valores de una clave multivalor

Caso real: escribiste `online` en vez de `oneline` en `alias.lg`. Se corrige volviendo a ejecutar `git config --global alias.lg "..."` con el valor bueno, y se sobrescribe.

3. Alias: `st`, `lg`, `last`

```
git config --global alias.st   "status -sb"
git config --global alias.lg   "log --graph --oneline --decorate --all"
git config --global alias.last "log -1 HEAD --stat"
```

Alias	Para qué
`git st`	Status corto con la rama arriba (`## main`)
`git lg`	Historial en grafo, una línea por commit, todas las ramas
`git last`	El último commit con los archivos que tocó

Status siempre es de la rama actual

`git status` / `git st` muestra la rama en la que estás (HEAD). Si cambias de rama, cambia lo que ves. Los cambios **sin commitear** viajan contigo al cambiar de rama.

Para mirar **otra** rama sin cambiarte a ella:

```
git log main..qa             # commits que tiene qa y main no
git diff --stat main develop # qué archivos difieren entre ramas
git show qa:archivo.txt      # un archivo tal como está en qa
git branch -vv               # todas las ramas con su último commit
```

4. Alias `hist` y formatos de log

```
git config --global alias.hist 'log --pretty=format:"%h %ad | %s%d [%an]" --date=short'
```

Marcador	Muestra	Marcador	Muestra
`%h`	hash corto	`%d`	ramas/tags, ej. `(HEAD -> main)`
`%ad`	fecha del autor	`%an`	nombre del autor
`%s`	asunto del commit	`--date=short`	fecha como `2025-01-01`

Tu salida:

```
7c91607 2026-09-23 | chore: prueba de autor local (HEAD -> main) [Jesus Practica]
a590b08 2025-01-01 | feat: despedida [Alumno]
541b555 2025-01-01 | chore: agrega .gitignore [Alumno]
```

💡 Pon el alias entre comillas **simples** para poder usar comillas dobles dentro. Los alias aceptan argumentos extra: `git hist -3` .

5. Ajustes globales recomendados

```
git config --global rerere.enabled true
git config --global rebase.autoSquash true
git config --global fetch.prune true
```

Ajuste	Qué hace	Módulo
`rerere.enabled`	Recuerda cómo resolviste un conflicto y lo repite solo	03
`rebase.autoSquash`	Junta automáticamente los commits `fixup!` / `squash!` en un rebase	02
`fetch.prune`	Al hacer fetch, borra las ramas remotas que ya no existen	07

Otros útiles: `pull.rebase true`, `rebase.autoStash true`, `merge.conflictStyle diff3`, `diff.colorMoved zebra`, `init.defaultBranch main`.

Git guarda las claves en minúsculas: `rebase.autosquash=true`.

6. Alias de shell (`!`): `ramas`

Si el alias empieza con `!`, se ejecuta como **comando de shell** y no como subcomando de git. Así puedes usar tuberías (`|`), `grep`, `xargs`...

```
git config --global alias.ramas '!git for-each-ref --sort=-committerdate refs/heads --
format="%(committerdate:relative)%09%(refname:short)"'

git ramas
# 45 minutes ago  develop
# 45 minutes ago  main
```

Las dos salían con la misma hora porque `develop` se creó sin commits propios: apuntaban al **mismo commit**.

7. `includeIf`: trabajo vs. personal

Problema: sin configurar nada, todos tus commits salen con el email global. En la empresa verían tu correo personal y GitHub/GitLab no asociaría esos commits a tu cuenta del trabajo.

Opción A: a mano, repo por repo

```
cd ~/trabajo/api-pagos
git config user.email "jesus@empresa.com"   # fácil de olvidar en cada clon nuevo
```

Opción B: automático por carpeta (recomendado)

```
# ~/.gitconfig
[user]
    name = Jesuslimon12
    email = correo-personal@gmail.com
[includeIf "gitdir/i:C:/Users/Jesus-Softmods/trabajo/"]
    path = ~/.gitconfig-trabajo

# ~/.gitconfig-trabajo
[user]
    email = jesus@empresa.com
```

- `gitdir/i`: no distingue mayúsculas y minúsculas (recomendado en Windows).
- La **barra final** `/` significa "cualquier repo dentro de esta carpeta". Usa `/`, no `\`.
- Solo aplica **dentro de repos**, porque compara la ruta de la carpeta `.git`.

Lo que hiciste en el ejercicio 7

```
git config --file ~/.gitconfig-practica user.email "tu-otro-correo@ejemplo.com"
git config --global includeIf."gitdir/i:C:/Users/Jesus-
Softmods/Documents/Proyectos_personales/curso_git_avanzado/laboratorio/".path "~/.gitconfig-
practica"
git config --unset --local user.email          # el local ganaba sobre el include
git config --show-origin user.email
# file:C:/Users/Jesus-Softmods/.gitconfig-practica   tu-otro-correo@ejemplo.com
```

⚠️ El **email** decide a qué cuenta de GitHub se **atribuye** el commit (y debe estar verificado en esa cuenta). **No** decide con qué cuenta haces **push**: eso depende de tus credenciales o tu clave SSH (`core.sshCommand`).

8. `git config --file` y dónde viven los archivos

`--file <ruta>` lee y escribe cualquier archivo con formato de config de Git. Si el archivo no existe, lo crea.

```
git config --file ~/mi-config user.email "algo@correo.com"   # crear/escribir
git config --file ~/mi-config --list                         # listar
git config --file ~/mi-config --unset user.email             # borrar clave
git config --global include.path "~/mi-config"               # cargarlo SIEMPRE
```

Crear el archivo **no lo activa**. Git solo lo lee si lo incluyes con `include.path` (siempre) o `includeIf` (con condición).

¿Se publica?

Depende de **dónde** lo crees, no del comando.

Ubicación	¿Se puede publicar?
Fuera de cualquier repo (`~/.gitconfig-practica`)	❌ Nunca
Dentro de `.git/`	❌ Nunca
Dentro del proyecto (`./.gitconfig-equipo`)	⚠️ Sí, si haces `add` + `commit` + `push`

`~` en Git Bash = `C:\Users\Jesus-Softmods`. Los archivos que empiezan con punto están ocultos en el Explorador (Vista → Mostrar → Elementos ocultos).

Archivos solo tuyos dentro de un repo

Archivo de reglas	¿Se comparte?
`.gitignore`	Sí, se commitea y lo recibe el equipo
`.git/info/exclude`	No, las reglas son solo tuyas

9. Reto: `deshacer` y `limpiar-ramas`

```
git config --global alias.deshacer "reset --soft HEAD~1"
git config --global alias.limpiar-ramas '!git branch --merged main | grep -vE "^\*|main$" |
xargs -r git branch -d'
```

Pieza	Qué hace
`reset --soft HEAD~1`	Quita el último commit; los cambios quedan en staging
`branch --merged main`	Lista las ramas ya fusionadas en `main`
`grep -vE "^\*|main$"`	Excluye `main` y la rama actual (`*`)
`xargs -r git branch -d`	Borra las que quedan (`-r`: nada si la lista está vacía)

```
git deshacer && git st
# ## main
# A  prueba.txt        ← A = added (en staging)

git limpiar-ramas
# Deleted branch vieja-1 (was 7c91607).
# Deleted branch vieja-2 (was 7c91607).
```

10. `restore` vs. `reset`

`restore` = **restaurar**: devuelve un archivo a una versión ya guardada. Trabaja con **archivos**. `reset` mueve la **rama** a otro commit: trabaja con el **historial**.

```
último commit (HEAD) ──restore --staged──► staging ──restore──► archivo de trabajo
```

Quieres volver atrás...	Comando	Copia de → a
Las **ediciones** que aún no agregaste	`git restore archivo`	staging → archivo
El `add`	`git restore --staged archivo`	HEAD → staging
El `commit`	`git reset --soft HEAD~1` (`git deshacer`)	mueve la rama
Todo al último commit	`git restore --source=HEAD --staged --worktree archivo`	HEAD → ambos
Un archivo a una versión antigua	`git restore --source=HEAD~2 archivo`	commit → archivo

⚠️ `git restore archivo` copia **desde el staging**, no desde el último commit. Por eso, con `prueba.txt` recién agregado, **no habría hecho nada**: el staging y el archivo tenían el mismo contenido.

```
echo "v1" > prueba.txt && git add prueba.txt   # staging: v1
echo "v2" > prueba.txt                          # archivo: v2
git restore prueba.txt                          # vuelve a v1; v2 se pierde
```

Para quitar un archivo **nuevo** por completo: `git restore --staged prueba.txt` y luego `rm prueba.txt`.

11. `HEAD~N` y `HEAD^`

El número después de `~` es **cuántos commits hacia atrás** cuentas.

Referencia	Commit (tu laboratorio)
`HEAD`	`7c91607` chore: prueba de autor local
`HEAD~1` = `HEAD~` = `HEAD^`	`a590b08` feat: despedida
`HEAD~2`	`541b555` chore: agrega .gitignore
`HEAD~3`	`e348de1` feat: agrega app

Funciona con cualquier referencia: `main~3`, `develop~1`, `7c91607~2`. `^` sirve además para elegir entre los padres de un merge (módulo 01).

⌨️ **Teclado mexicano:** `~` = **Alt + 126** en el teclado numérico (con Bloq Num activado).

12. Chuleta final

Alias	Comando real
`git st`	`status -sb`
`git lg`	`log --graph --oneline --decorate --all`
`git last`	`log -1 HEAD --stat`
`git hist`	`log --pretty=format:"%h %ad | %s%d [%an]" --date=short`
`git ramas`	`!git for-each-ref --sort=-committerdate refs/heads ...`
`git deshacer`	`reset --soft HEAD~1`
`git limpiar-ramas`	`!git branch --merged main | grep -vE ... | xargs -r git branch -d`

Ver todos tus alias: `git config --global --get-regexp ^alias`

Preguntas de repaso

1. ¿Qué nivel gana si `user.email` está en global y en local?
2. ¿Qué hace `merge.conflictStyle diff3` y por qué ayuda?
3. ¿Por qué un alias que empieza con `!` se comporta distinto?
4. ¿Por qué en el ejercicio 7 hubo que borrar el `user.email` local?
5. ¿Qué diferencia hay entre `git restore archivo` y `git restore --staged archivo`?
6. ¿`git config --file` publica el archivo que crea?

Respuestas: (1) El local, porque es el más específico. (2) En los conflictos muestra también la versión del ancestro común (`|||||||`), así se ve qué cambió cada lado. (3) Se ejecuta como comando de shell, lo que permite usar tuberías y otros programas. (4) Porque el nivel local gana sobre el global, y el `includeIf` se carga como parte del global. (5) La primera descarta las ediciones del archivo (vuelve a lo del staging); la segunda saca el archivo del staging (deshace el `add`). (6) No. Solo se publica si está dentro del proyecto y haces `add`, `commit` y `push`.